ITCS525 Lecture 2

Testing, GitHub, and CI/CD

Testing

A critical part of the software development process is testing the code we've written to ensure everything runs as expected. In this lecture, we'll discuss several ways to improve the way we test our code.

Testing Manually using Assert

A minimum test that we all **must** do is to test our code manually.

In Python, we can use `assert` command.

```
assert expression[, assertion_message]
```

If the `expression` is `True`, nothing happens

If the `expression` is `False`, an *exception* will be thrown.



For example, we can test whether a `square(...)` function is working or not by using `assert` statement(s).

```
def square(x):
    return x * x

assert square(10) == 100

""" Output:

"""
```

```
def square(x):
    return x + x

assert square(10) == 100

""" Output:
Traceback (most recent call last):
  File "assert.py", line 4, in <module>
    assert square(10) == 100
AssertionError
"""
```

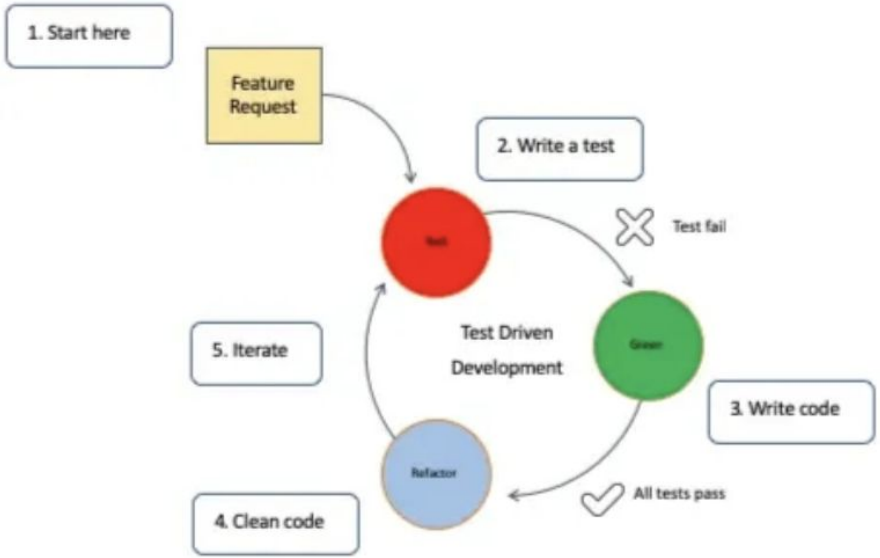
💡 *What is an exception?*

Errors detected during execution are called **exceptions** and are not unconditionally fatal.

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
         ~^~
ZeroDivisionError: division by zero
```

Test-driven development

Test-driven development (TDD) is a method of writing code that involves writing an automated unit-level test case that fails, then writing just enough code to make the test pass, and then refactoring both the test code and the production code. This process is repeated with another new test case.



https://developer.ibm.com/articles/5-steps-of-test-driven-development/

Testing automatically using a script

As we build a larger project, manual testing would take a significant amount of time.

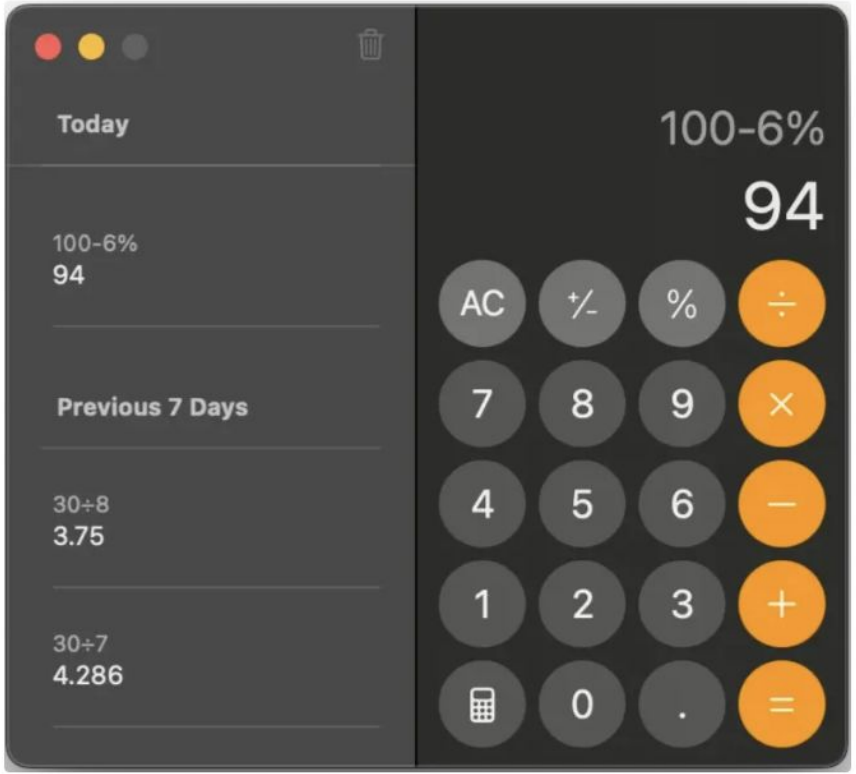
- Once you have finished coding, you need to test it.
- When you fix bugs in your code, you need to retest it.
- Whenever someone changes the code, we need to retest it.
- ...

☝️ Testing is critical in software development, such that there is a career for it.

"**A QA (Quality Assurance) Engineer** ensures software quality by designing, developing, and executing tests, identifying defects, and collaborating with developers to improve product functionality."

Let's say we want to build a simple calculator application (albeit a client-server architecture).

We have to be able to expand an expression like `20 - 6%` to `20 - 6/100 * 20`.



```python
# calculator.py
import re

_percent_pair = re.compile(r"""
    (?P<a>\d+(?:\.\d+)?)
    \s*(?P<op>[+\-*/])\s*
    (?P<b>\d+(?:\.\d+)?)%
""", re.VERBOSE)

def expand_percent(expr: str) -> str:
    """Handle A op B% and standalone N% patterns."""
    s = expr
    while True:
        # Replace A op B%
        m = _percent_pair.search(s)
        if not m:
            break
        a, op, b = m.group("a", "op", "b")
        if op in "+-":
            repl = f"{a} {op} (({b}/100)*{a})"
        elif op == "*":
            repl = f"{a} * ({b}/100)"
        else:
            repl = f"{a} / ({b}/100)"
        s = s[:m.start()] + repl + s[m.end():]

    return s
```

Now, let's take a look at a function we've written to test our `expand_percent(.)` function:

```
# test-calculator.py
from calculator import expand_percent

def test_expand_percent():
    assert expand_percent("5 + 10%") == "5 + ((10/100)*5)", (
        'Failed on 5 + 10%, result: ' + expand_percent("5 + 10%"))
    assert expand_percent("20 - 30%") == "20 - ((30/100)*20)", (
        'Failed on 20 - 30%, result: ' + expand_percent("20 - 30%"))
    assert expand_percent("15 * 25%") == "15 * (25/100)", (
        'Failed on 15 * 25%, result: ' + expand_percent("15 * 25%"))
    assert expand_percent("40 / 50%") == "40 / (50/100)",(
        'Failed on 40 / 50%, result: ' + expand_percent("40 / 50%"))
    assert expand_percent("3 * 4% + 2 / 1%") == "3 * (4/100) + 2 / (1/100)", (
        'Failed on 3 * 4% + 2 / 1%, result: '+expand_percent("3 * 4% + 2 / 1%"))
    assert expand_percent("100%") == "(100/100)", (
        'Failed on 100%, result: ' + expand_percent("100%"))
    assert expand_percent("10% + 20%") == "(10/100) + (20/100)", (
        'Failed on 10% + 20%, result: ' + expand_percent("10% + 20%"))

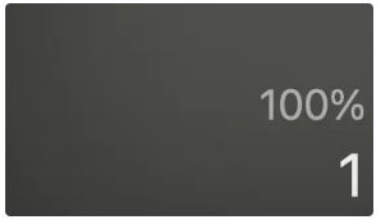
if __name__ == "__main__":
    test_expand_percent()
    print("All tests passed!")
```

We can run the tests by executing this Python file:

```
python test-calulator.py
```

```
Traceback (most recent call last):
  File "test-calculator.py", line 20, in <module>
    test_expand_percent()
    ~~~~~~~~~~~~~~~~~~~~^^
  File "test-calculator.py", line 14, in test_expand_percent
    assert expand_percent("100%") == "(100/100)", (
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: Failed on 100%, result: 100%
```

The test fails at line 14, which tests `100%`.
We expect `(100/100)`, but the function returns `100%`.



🧪 Can you fix it?

Aside: Python `import`

Assuming that we have `calculator.py` and `test-calculator.py` in the same folder. We can import the function from `calculator.py` to `test-calculator.py` in 3 different ways.

1. `import MODULE`: This method imports the module into the namespace under the module's name.

```
import calculator
calculator.expand_percent('100%')
```

2. `from MODULE import XXX`: This method imports items from within the module directly into the program's main namespace.

```
from calculator import expand_percent
expand_percent('100%')
```

3. `import MODULE as yyy`: This method imports the module into the namespace under the name `yyy`.

```
import calculator as cal
cal.expand_percent('100%')
```

Unit Testing Framework: `pytest`

Unit testing frameworks, such as `unittest` or `pytest`, make the testing easier.

1. Automatic test discovery and run all tests at once in one command, instead of having to write `main()` and run each test file.
2. Better error reporting and no need for print debugging

`unittest` is Python's built-in library for testing, but we will use `pytest` as FastAPI documentation recommends `pytest`.

```
pip install pytest
```

`pytest` A quick example

pytest documentation

```python
# content of test_sample.py
def inc(x):
    return x + 1


def test_answer():
    assert inc(3) == 5
```

```
$ pytest
============================= test session starts ==============================
platform linux -- Python 3.x.y, pytest-8.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_sample.py F                                                        [100%]

================================= FAILURES =================================
_____________________________________ test_answer _____________________________________

    def test_answer():
>       assert inc(3) == 5
E       assert 4 == 5
E        +  where 4 = inc(3)

test_sample.py:6: AssertionError
========================== short test summary info ==========================
FAILED test_sample.py::test_answer - assert 4 == 5
============================== 1 failed in 0.12s ===============================
```

Let's write unit tests for our `expand_percent(.)` function. By default, `pytest` will look for `test_xxx.py` files for testing.

```
#test_calculator.py

import pytest
from calculator import expand_percent  # import your function

def test_add_percent():
    """Test addition where B% means 'B percent of A'."""
    assert expand_percent("5 + 10%") == "5 + ((10/100)*5)"

def test_subtract_percent():
    """Test subtraction where B% means 'B percent of A'."""
    assert expand_percent("20 - 30%") == "20 - ((30/100)*20)"
```

```python
def test_multiply_percent():
    """Test multiplication where B% means 'B divided by 100'."""
    assert expand_percent("15 * 25%") == "15 * (25/100)"

def test_divide_percent():
    """Test division where B% means 'B divided by 100'."""
    assert expand_percent("40 / 50%") == "40 / (50/100)"

def test_multiple_operations():
    """Test expressions with multiple A op B% operations."""
    assert expand_percent("3 * 4% + 2 / 1%") == "3 * (4/100) + 2 / (1/100)"

def test_standalone_100_percent():
    """Test standalone percentage (100%)."""
    assert expand_percent("100%") == "(100/100)"

def test_two_standalone_percents():
    """Test expression with two standalone percentages."""
    assert expand_percent("10% + 20%") == "(10/100) + (20/100)"
```

```
nor@local ~/L/C/G/M/I/I/I/l/calulator-server> pytest -v                    (itcs525)
================================= test session starts ==================================
platform darwin -- Python 3.13.5, pytest-8.4.1, pluggy-1.6.0 -- /Users/northanapon/miniconda3/
envs/itcs525/bin/python3.1
cachedir: .pytest_cache
rootdir: /Users/northanapon/Library/CloudStorage/GoogleDrive-thanapon.nor@mahidol.ac.th/My Dri
ve/ICT-Teaching/ITCS525/ITCS525-Code/lecture-02/calulator-server
plugins: anyio-4.9.0
collected 7 items

test_calculator.py::test_add_percent PASSED                                [ 14%]
test_calculator.py::test_subtract_percent PASSED                           [ 28%]
test_calculator.py::test_multiply_percent PASSED                           [ 42%]
test_calculator.py::test_divide_percent PASSED                             [ 57%]
test_calculator.py::test_multiple_operations PASSED                        [ 71%]
test_calculator.py::test_standalone_100_percent FAILED                     [ 85%]
test_calculator.py::test_two_standalone_percents FAILED                    [100%]

======================================== FAILURES ========================================
_____________________________ test_standalone_100_percent ______________________________

    def test_standalone_100_percent():
        """Test standalone percentage (100%)."""
>       assert expand_percent("100%") == "(100/100)"
E       AssertionError: assert '100%' == '(100/100)'
E
E         - (100/100)
E         + 100%

test_calculator.py:26: AssertionError
_____________________________ test_two_standalone_percents ______________________________

    def test_two_standalone_percents():
        """Test expression with two standalone percentages."""
>       assert expand_percent("10% + 20%") == "(10/100) + (20/100)"
E       AssertionError: assert '10% + 20%' == '(10/100) + (20/100)'
E
E         - (10/100) + (20/100)
E         + 10% + 20%

test_calculator.py:30: AssertionError
================================= short test summary info ==================================
FAILED test_calculator.py::test_standalone_100_percent - AssertionError: assert '100%' == '(10
0/100)'
FAILED test_calculator.py::test_two_standalone_percents - AssertionError: assert '10% + 20%' =
= '(10/100) + (20/100)'
================================= 2 failed, 5 passed in 0.05s ==============================
nor@local ~/L/C/G/M/I/I/I/l/calulator-server>                              (itcs525)
```

FastAPI Testing

Let’s take a look at how we can test a backend server using what we have learned so far.

Running example

Let's assume we have a file named `main.py` in our folder.

```
import math

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from asteval import Interpreter

from calculator import expand_percent

app = FastAPI(title="Mini Calculator API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```python
aeval = Interpreter(minimal=True, usersyms={"pi": math.pi, "e": math.e})

@app.post("/calculate")
def calculate(expr: str):
    try:
        code = expand_percent(expr)
        result = aeval(code)
        if aeval.error:
            msg = "; ".join(str(e.get_error()) for e in aeval.error)
            aeval.error.clear()
            return {"ok": False, "expr": expr, "result": "", "error": msg}
        return {"ok": True, "expr": expr, "result": result, "error": ""}
    except Exception as e:
        return {"ok": False, "expr": expr, "error": str(e)}
```

Aside: `asteval`

The `asteval` is a library that we can use to evaluate an expression like `1 - 2 + 10` in Python code format.

```
pip install asteval
```

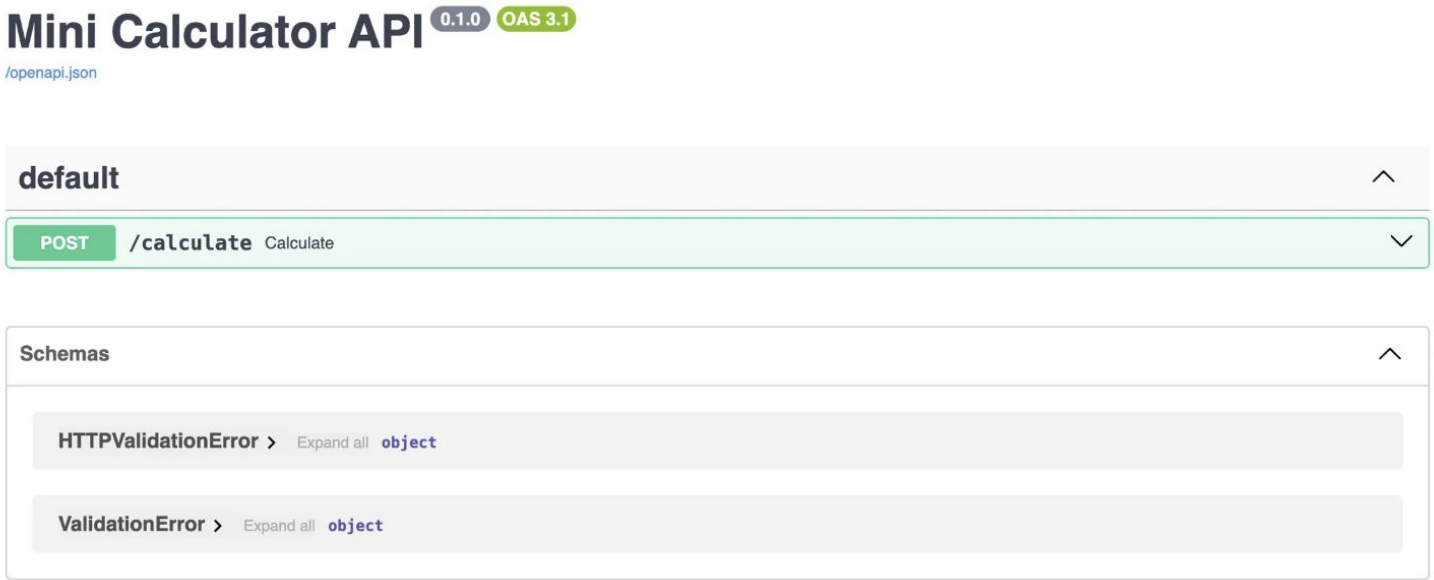
```
from asteval import Interpreter
aeval = Interpreter()
aeval('42 * 2 + 2')
# 86
```

Manually testing FastAPI

You can access the FastAPI swagger document to test the APIs without using additional tools.

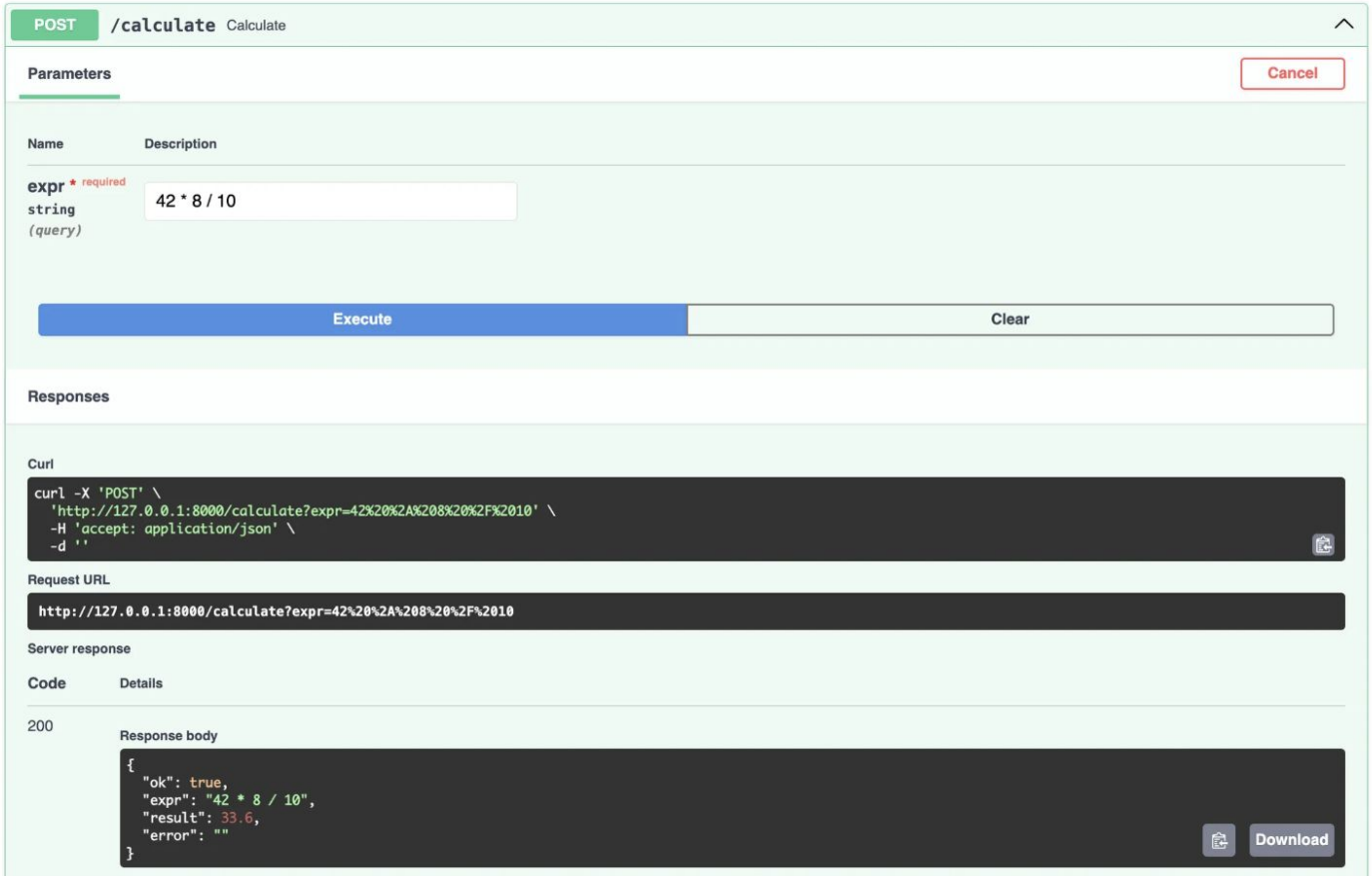
```
fastapi dev main.py
```

Go to http://127.0.0.1:8000/docs



To test an API, we can click on the “Try it out”.





If we test enough cases, we will find a bug with the input: `5%`.

Response body


```
{
  "ok": false,
  "expr": "5%",
  "result": "",
  "error": "('SyntaxError', '5%\\nSyntaxError: invalid syntax (<unknown>, line 1)')"
}
```


Download

Using `TestClient`

We can automate testing our APIs using `TestClient` library of FastAPI. To do this, we first have to install `HTTPX` library first.

```
pip install httpx
```

uv add httpx2

Let's create `test_main.py` under the same folder as `main.py` with the following content:

```
from fastapi.testclient import TestClient
from main import app  # or whatever your app module is

client = TestClient(app)

def test_basic_division():
    r = client.post("/calculate", params={"expr": "30/4"})
    assert r.status_code == 200
    data = r.json()
    assert data["ok"] is True
    assert abs(data["result"] - 7.5) < 1e-9

def test_percent_subtraction():
    r = client.post("/calculate", params={"expr": "100 - 6%"})
    assert r.status_code == 200
    data = r.json()
    assert data["ok"] is True
    assert abs(data["result"] - 94.0) < 1e-9
```

```python
def test_standalone_percent():
    r = client.post("/calculate", params={"expr": "6%"})
    assert r.status_code == 200
    data = r.json()
    assert data["ok"] is True
    assert abs(data["result"] - 0.06) < 1e-9

def test_invalid_expr_returns_ok_false():
    r = client.post("/calculate", params={"expr": "2**(3"})
    assert r.status_code == 200
    data = r.json()
    assert data["ok"] is False
    assert "error" in data and data["error"] != ""
```

```
nor@local ~/L/C/G/M/I/I/I/l/calulator-server> pytest -v test_main.py
========================== test session starts ==========================
platform darwin -- Python 3.13.5, pytest-8.4.1, pluggy-1.6.0 -- /Users/north
anapon/miniconda3/envs/itcs525/bin/python3.1
cachedir: .pytest_cache
rootdir: /Users/northanapon/Library/CloudStorage/GoogleDrive-thanapon.nor@ma
hidol.ac.th/My Drive/ICT-Teaching/ITCS525/ITCS525-Code/lecture-02/calulator-
server
plugins: anyio-4.9.0
collected 4 items

test_main.py::test_basic_division PASSED                                 [ 25%]
test_main.py::test_percent_subtraction PASSED                            [ 50%]
test_main.py::test_standalone_percent FAILED                             [ 75%]
test_main.py::test_invalid_expr_returns_ok_false PASSED                  [100%]

================================== FAILURES ==================================
_____________________________ test_standalone_percent _____________________________

    def test_standalone_percent():
        r = client.post("/calculate", params={"expr": "6%"})
        assert r.status_code == 200
        data = r.json()
>       assert data["ok"] is True
E       assert False is True

test_main.py:24: AssertionError
---------------------------- Captured stderr call -----------------------------
6%
SyntaxError: invalid syntax (<unknown>, line 1)
======================== short test summary info ========================
FAILED test_main.py::test_standalone_percent - assert False is True
====================== 1 failed, 3 passed in 0.61s ======================
nor@local ~/L/C/G/M/I/I/I/l/calulator-server>                    (itcs525)
```

Okay, let's fix our `expand_precent(.)` function

```
import re
_percent_pair = re.compile(r"""
    (?P<a>\d+(?:\.\d+)?)
    \s*(?P<op>[+\-*/])\s*
    (?P<b>\d+(?:\.\d+)?)%
""", re.VERBOSE)
_number_percent = re.compile(r"(?P<n>\d+(?:\.\d+)?)%")

def expand_percent(expr: str) -> str:
    """Handle A op B% and standalone N% patterns."""
    s = expr
    while True:
        # Replace A op B%
        m = _percent_pair.search(s)
        if not m:
            break
        a, op, b = m.group("a", "op", "b")
        if op in "+-":
            repl = f"{a} {op} (({b}/100)*{a})"
        elif op == "*":
            repl = f"{a} * ({b}/100)"
        else:
            repl = f"{a} / ({b}/100)"
        s = s[:m.start()] + repl + s[m.end():]

    # Replace B%
    s = _number_percent.sub(lambda m: f"({m.group('n')}/100)", s)
    return s
```

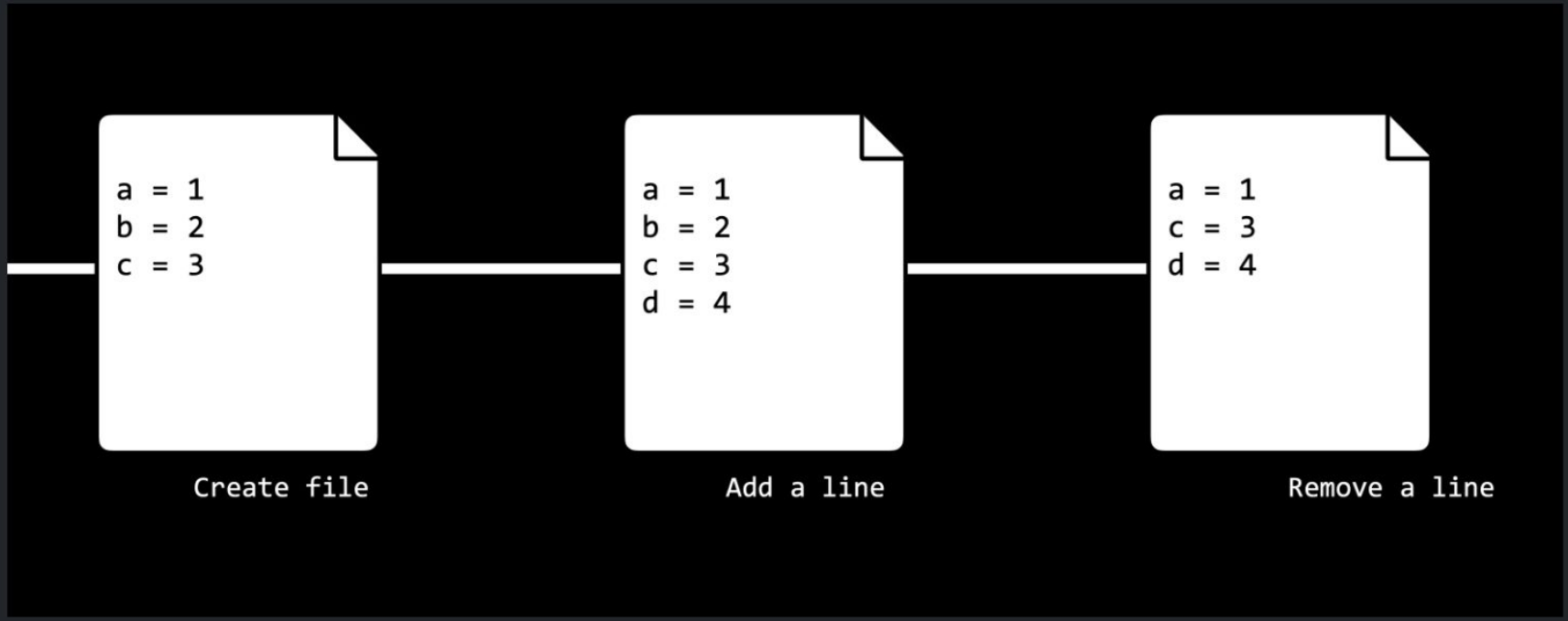
```
● nor@local ~/L/C/G/M/I/I/I/l/calulator-server> pytest -v          (itcs525)
============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-8.4.1, pluggy-1.6.0 -- /Users/north
anapon/miniconda3/envs/itcs525/bin/python3.1
cachedir: .pytest_cache
rootdir: /Users/northanapon/Library/CloudStorage/GoogleDrive-thanapon.nor@ma
hidol.ac.th/My Drive/ICT-Teaching/ITCS525/ITCS525-Code/lecture-02/calulator-
server
plugins: anyio-4.9.0
collected 11 items

test_calculator.py::test_add_percent PASSED                              [  9%]
test_calculator.py::test_subtract_percent PASSED                         [ 18%]
test_calculator.py::test_multiply_percent PASSED                         [ 27%]
test_calculator.py::test_divide_percent PASSED                           [ 36%]
test_calculator.py::test_multiple_operations PASSED                      [ 45%]
test_calculator.py::test_standalone_100_percent PASSED                   [ 54%]
test_calculator.py::test_two_standalone_percents PASSED                  [ 63%]
test_main.py::test_basic_division PASSED                                 [ 72%]
test_main.py::test_percent_subtraction PASSED                            [ 81%]
test_main.py::test_standalone_percent PASSED                             [ 90%]
test_main.py::test_invalid_expr_returns_ok_false PASSED                  [100%]

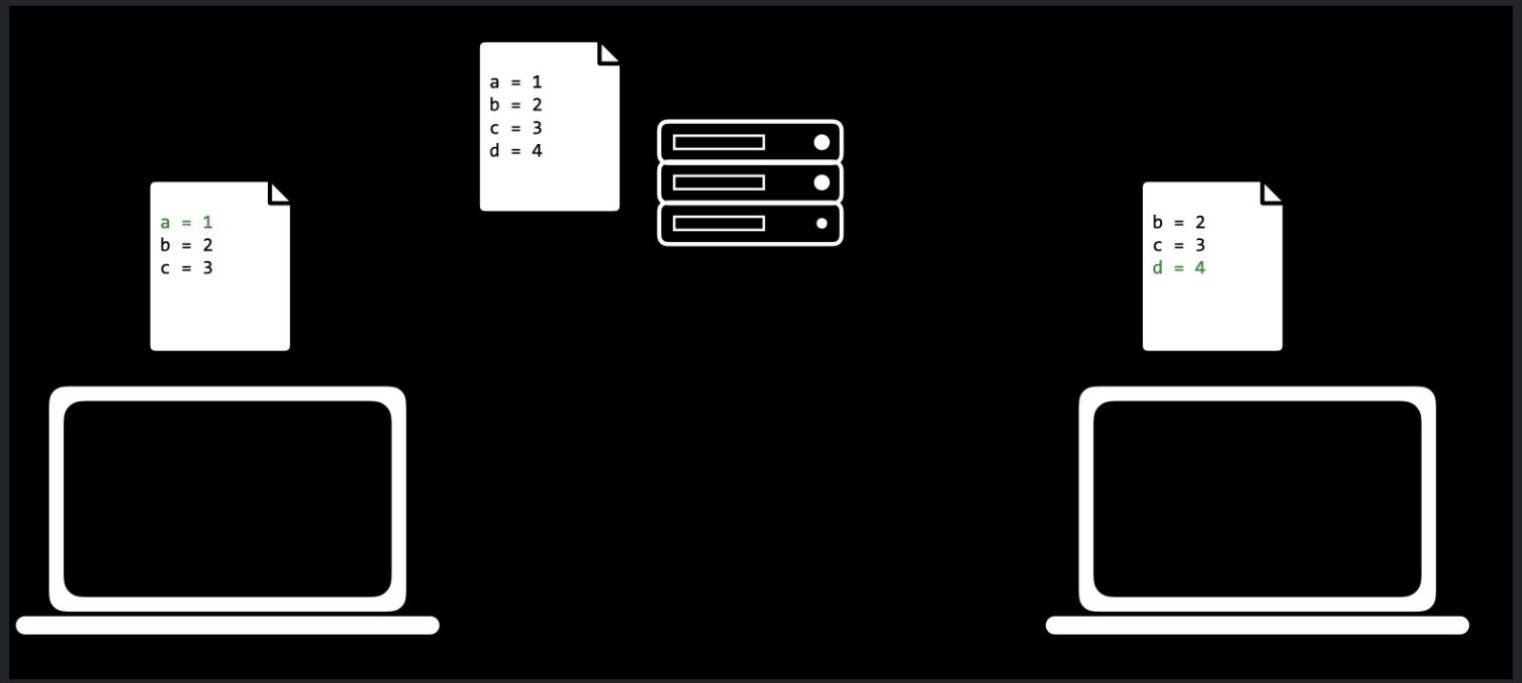
============================== 11 passed in 0.26s ==============================
● nor@local ~/L/C/G/M/I/I/I/l/calulator-server>                   (itcs525)
```

Git

- Git is a command line tool that will help us with version control in several different ways:
  - Allowing us to keep track of changes we make to our code by saving snapshots of our code at a given point in time.



- Allowing us to easily synchronize code between different people working on the same project by allowing multiple people to pull information from and push information to a repository stored on the web.



- Allowing us to make changes to and test out code on a different *branch* without altering our main code base, and then merging the two together.
- Allowing us to revert back to earlier versions of our code if we realize we've made a mistake.

GitHub

- GitHub is a website that allows us to store Git repositories remotely on the web.

If you have not yet had a GitHub account, you must sign up for one to continue.

Commits

- Now, we'll start to get into what Git can be really useful for. After making some changes to a file, we can *commit* those changes, taking a snapshot of the current state of our code. To do this, we run: `git commit -m "some message"` where the message describes the changes you just made.
- After this change, we can run `git status` to see how our code compares to the code on the remote repository
- When we're ready to publish our local commits to Github, we can run `git push`. Now, when we go to GitHub in our web browser, our changes will be reflected.
- If you've only changed existing files and not created new ones, instead of using `git add .` and then `git commit...`, we can condense this into one command: `git commit -am "some message"`. This command will commit all the changes that you made.
- Sometimes, the remote repository on GitHub will be more up to date than the local version. In this case, you want to first commit any changes, and then run `git pull` to pull any remote changes to your repository.

Check status

```
● nor@local ~/L/C/G/M/I/I/I/l/calculator-server> git status                    (base)
  On branch main
  Changes not staged for commit:
    (use "git add <file>..." to update what will be committed)
    (use "git restore <file>..." to discard changes in working directory)
          modified:   main.py

  Untracked files:
    (use "git add <file>..." to include in what will be committed)
          __pycache__/

  no changes added to commit (use "git add" and/or "git commit -a")
```

Commit

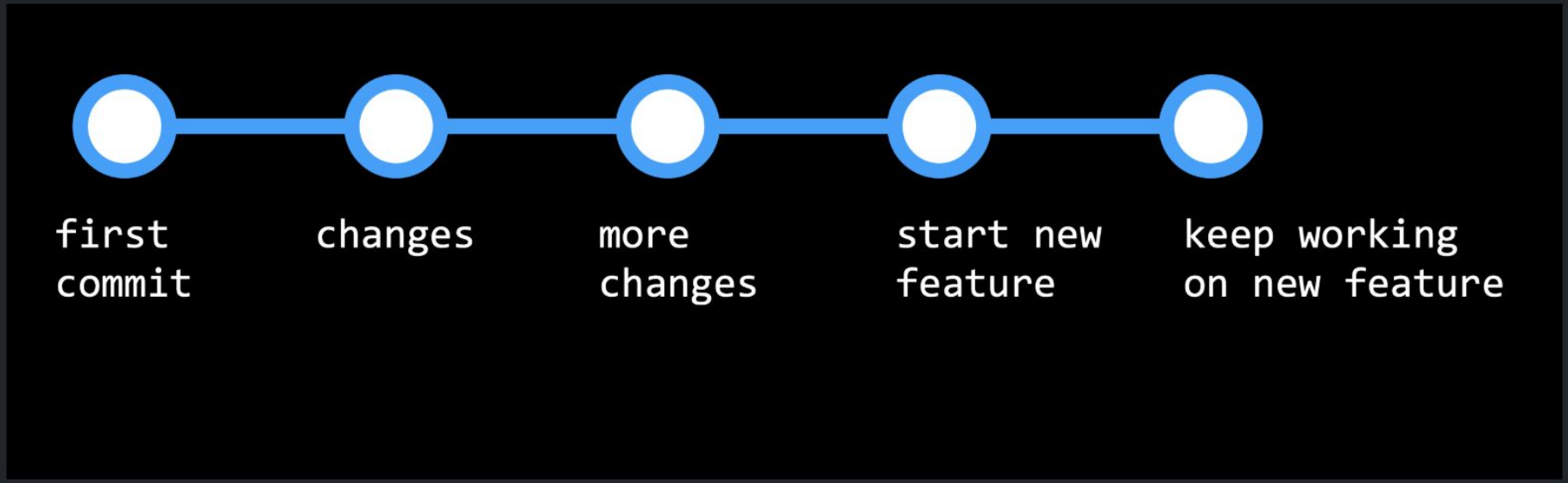
```
● nor@local ~/L/C/G/M/I/I/I/l/calculator-server> git commit -a -m "Added /history (GET and DELETE)"  (base)
  [main 3a0270c] Added /history (GET and DELETE)
   1 file changed, 20 insertions(+), 1 deletion(-)
```

Push

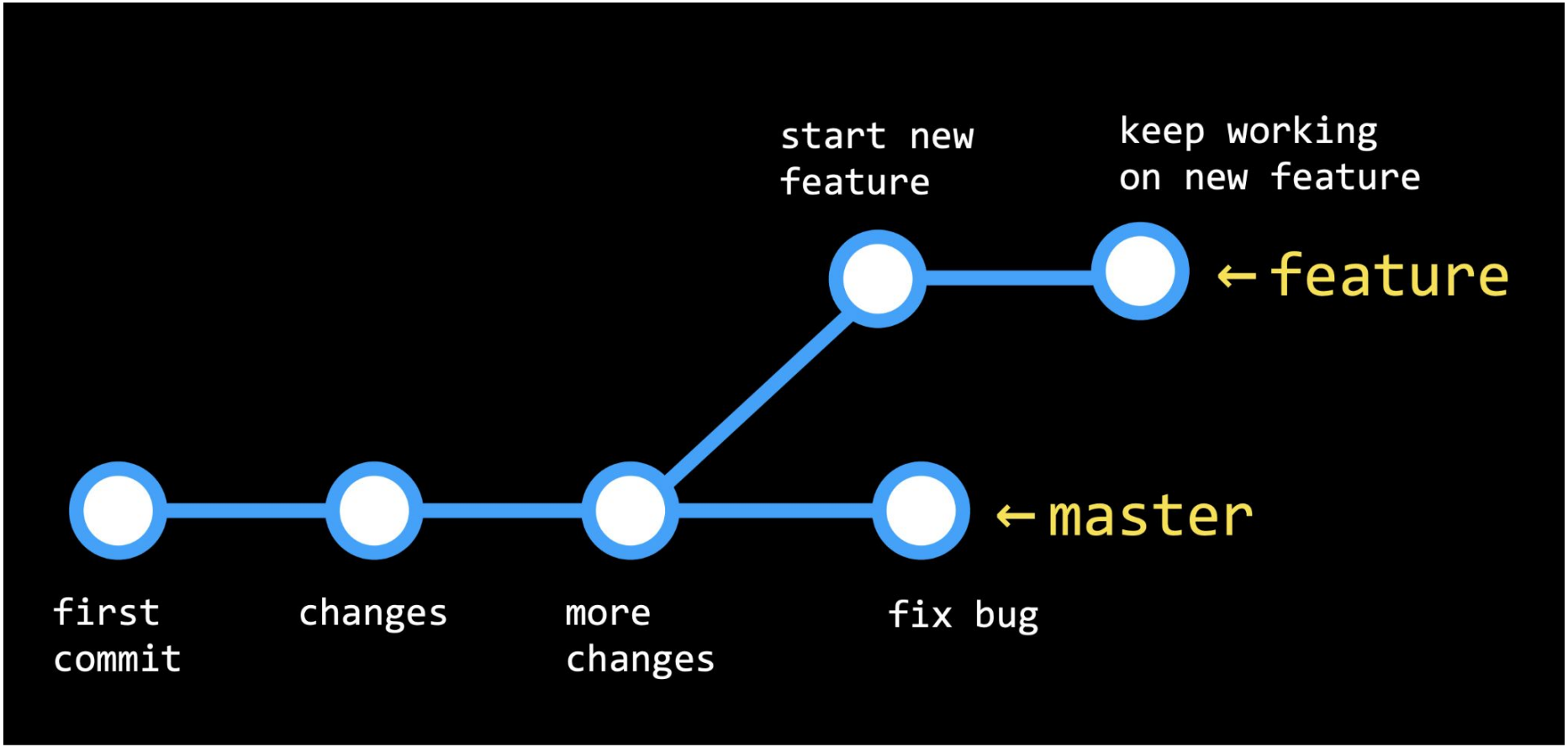
```
● nor@local ~/L/C/G/M/I/I/I/l/calculator-server> git push origin main         (base)
  Enumerating objects: 5, done.
  Counting objects: 100% (5/5), done.
  Delta compression using up to 10 threads
  Compressing objects: 100% (3/3), done.
  Writing objects: 100% (3/3), 615 bytes | 615.00 KiB/s, done.
  Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
  remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
  To https://github.com/ICT-Mahidol/ITCS525-Calculator-Server.git
     2f9ce3c..3a0270c  main -> main
```

Branching

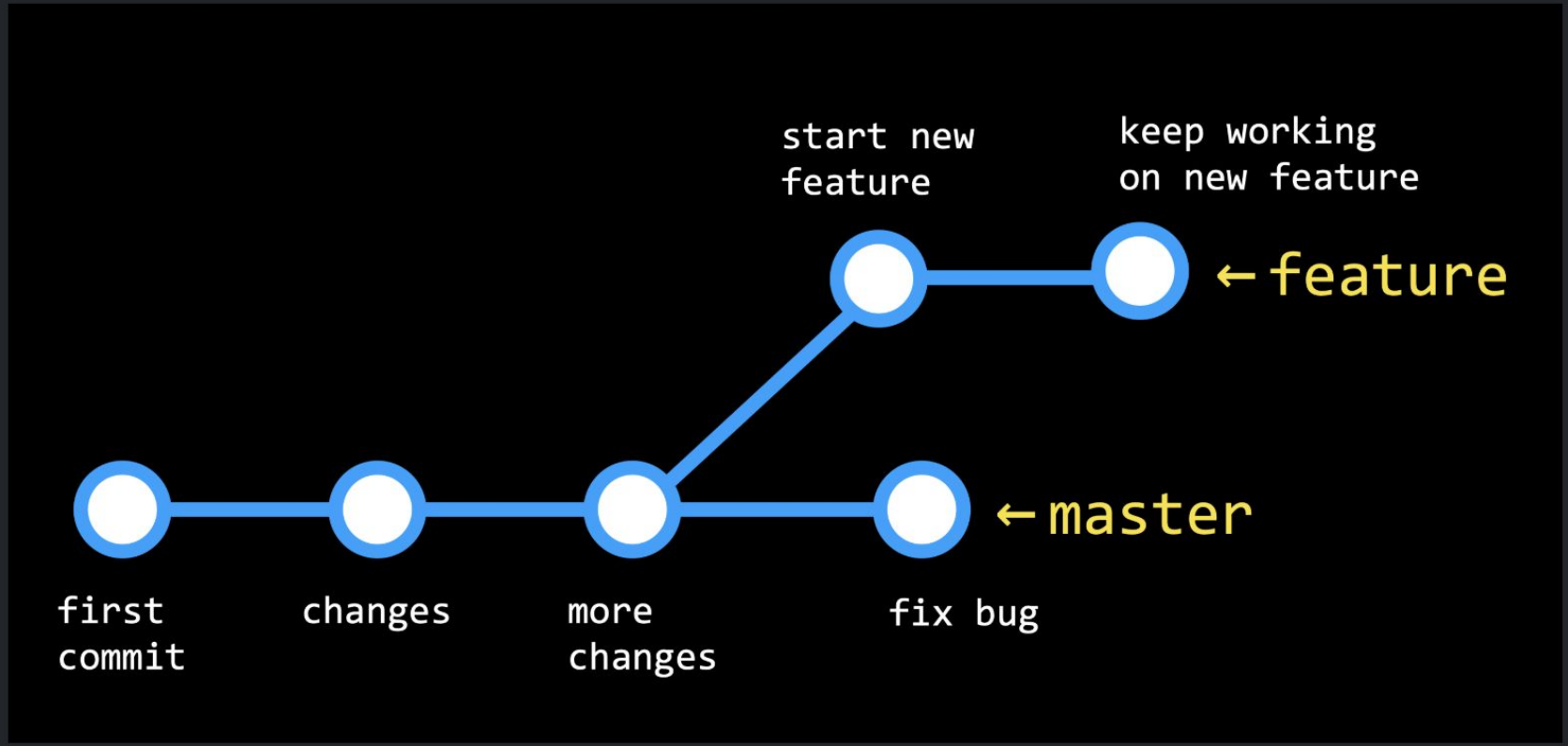
After you've been working on a project for some time, you may decide that you want to add an additional feature. At the moment, we may just commit changes to this new feature as shown in the graphic below



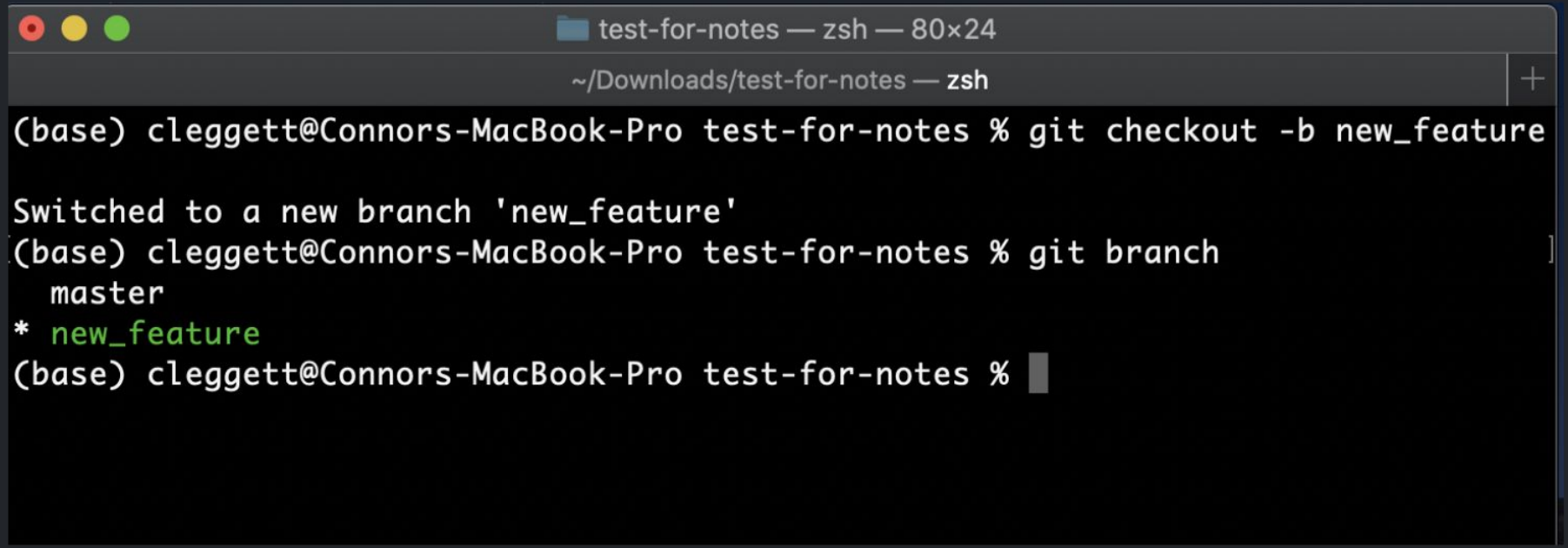
But this could become problematic if we then discover a bug in our original code, and want to revert back without changing the new feature. This is where branching can become really useful.


start new feature
keep working on new feature
← feature
← master
first commit
changes
more changes
fix bug

- Branching is a method of moving into a new direction when creating a new feature, and only combining this new feature with the main part of your code, or the main branch, once you're finished. This workflow will look more like the below graphic:

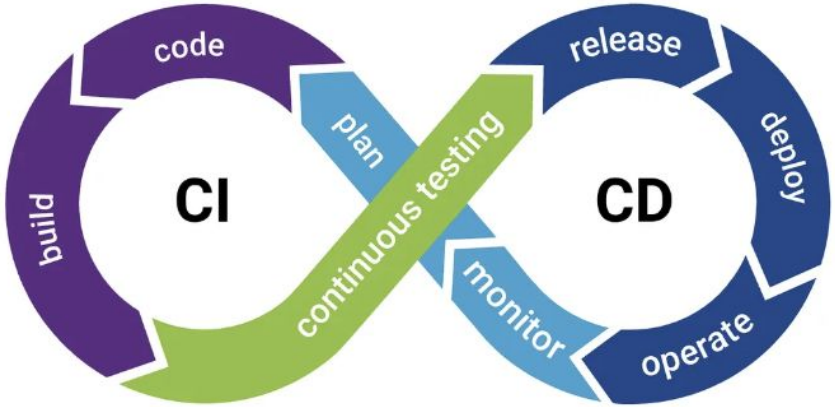


1. To make a new branch, we'll run `git checkout -b <new branch name>`



Continuous Integration and Continuous Delivery

CI/CD is a set of software development best practices that dictate how a team writes code and how that code is subsequently delivered to the application's users.



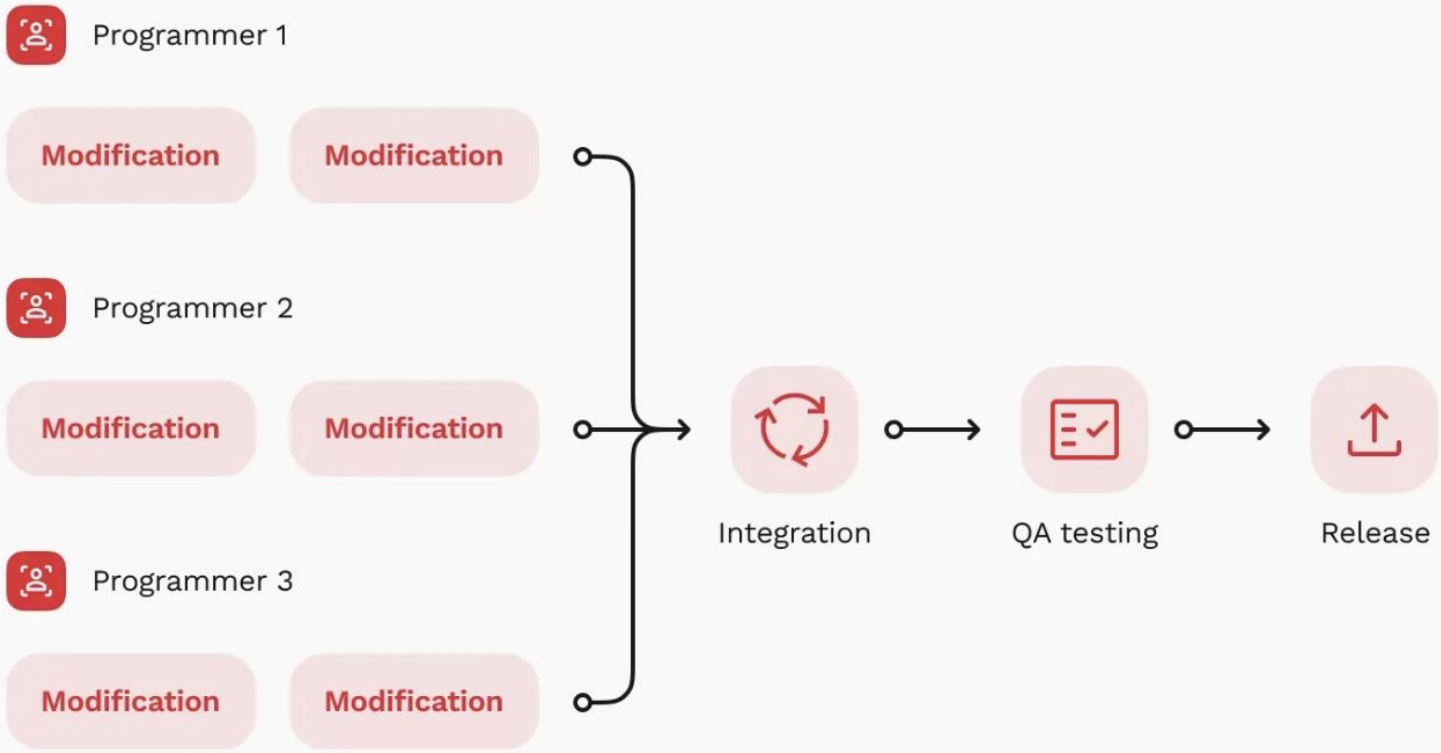
CI/CD

- Continuous Integration:
  - Frequent merges to the main branch
  - Automated unit testing with each merge
- Continuous Delivery:
  - Short release schedules, meaning new versions of an application are released frequently.

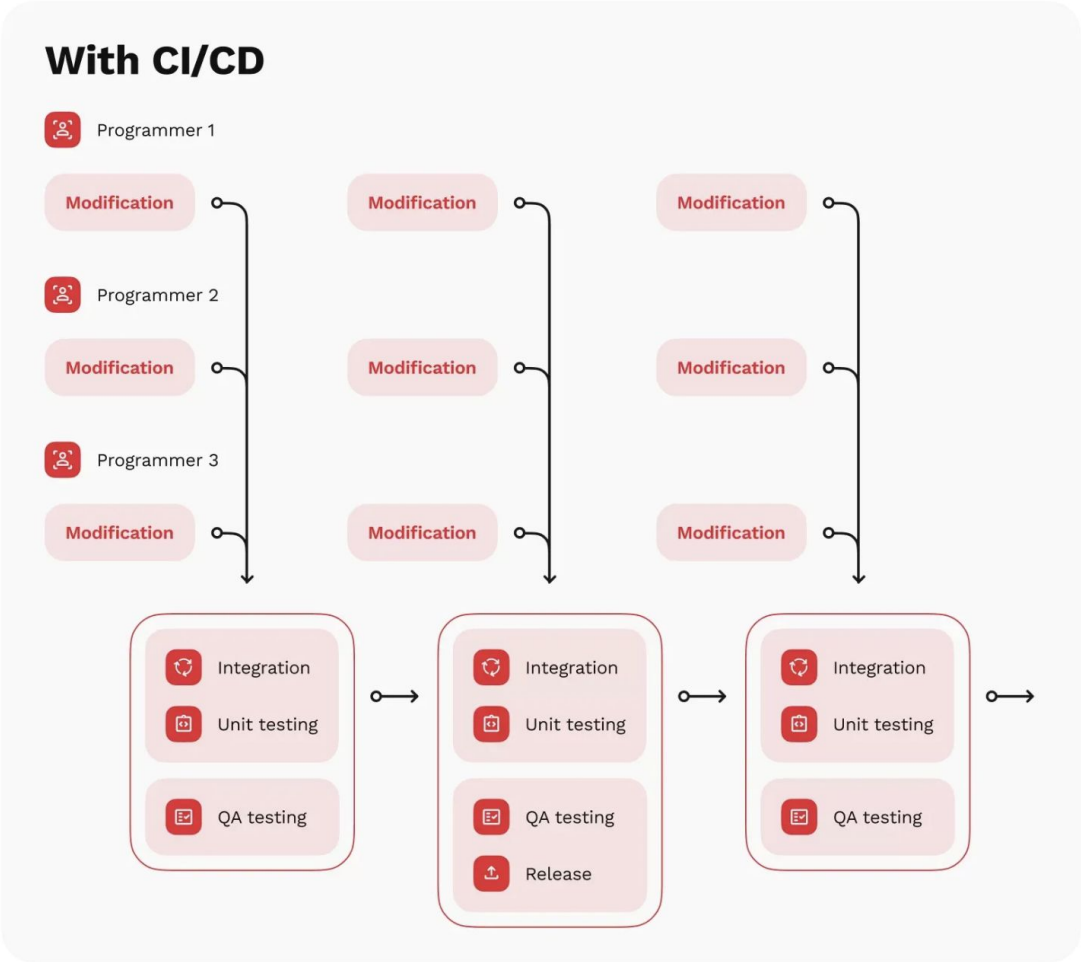
☝️ In this lecture, we will focus only on CI.

We will get back to CD later, when we learn about Docker and Containerization.

Without CI/CD



https://fivedottwelve.com/blog/why-ci-cd-is-a-must-have-in-your-software-project/



https://fivedottwelve.com/blog/why-ci-cd-is-a-must-have-in-your-software-project/

Why?

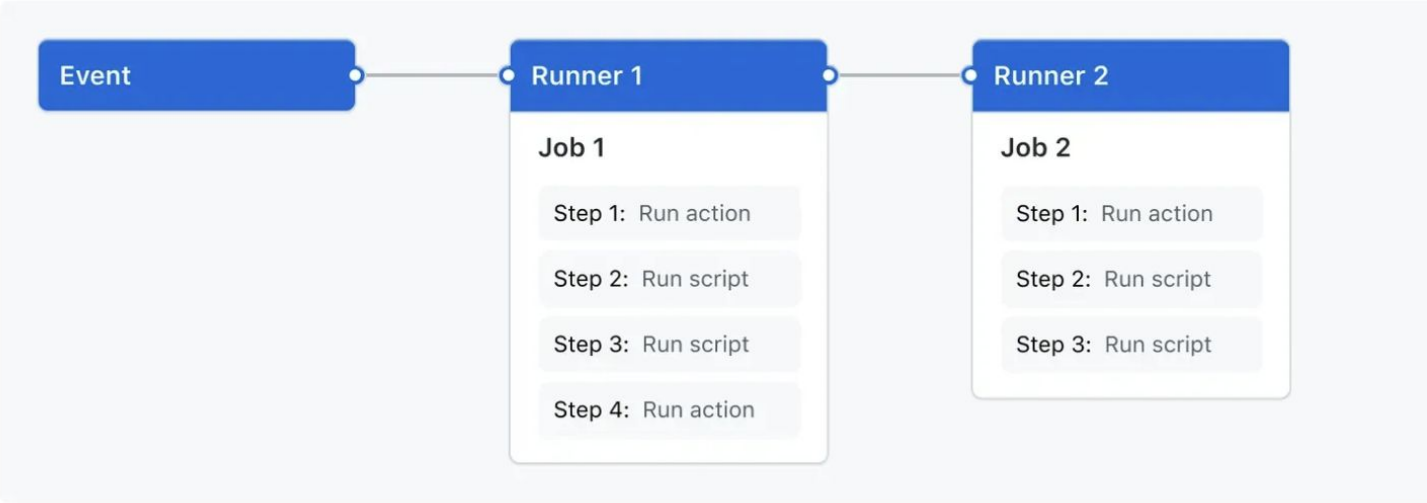
CI/CD is crucial because it **automates the software development process from coding through deployment**.

- Release new features and fixes more quickly and frequently, making products more responsive to user needs.
- Detect errors earlier, which reduces downtime and improves software quality.
- Create quicker feedback loops with stakeholders, ensuring the final product closely aligns with user expectations.

Github Actions for CI/CD

Ref: GitHub Docs Understanding GitHub Actions - GitHub Docs

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform.

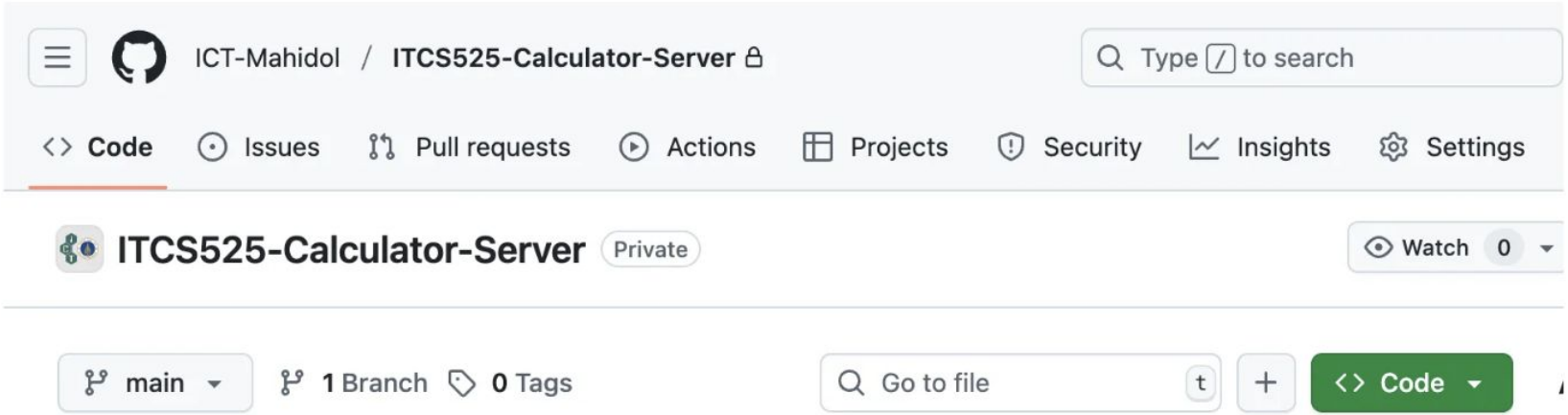


You can configure a GitHub Actions **workflow** to be triggered when an **event** occurs in your repository, such as a pull request being opened or an issue being created. Your workflow contains one or more **jobs** that can run in sequential order or parallel. Each job will run inside its virtual machine **runner** or inside a container and has one or more **steps** that either run a script you define or execute an **action**, which is a reusable extension that can simplify your workflow.

The CI Workflow

Ref: GitHub Docs [Building and testing Python - GitHub Docs]

1. On GitHub, navigate to the repository's main page.
2. Under your repository name, click **Actions**.



3. Click **"set up a workflow yourself"**.
4. Configure the workflow using YAML format.
5. Click **Commit changes.**

YAML

To set up a GitHub action, we'll use a configuration language called **YAML**. YAML structures its data around key-value pairs (like a JSON object or Python Dictionary). Here's an example of a simple YAML file:

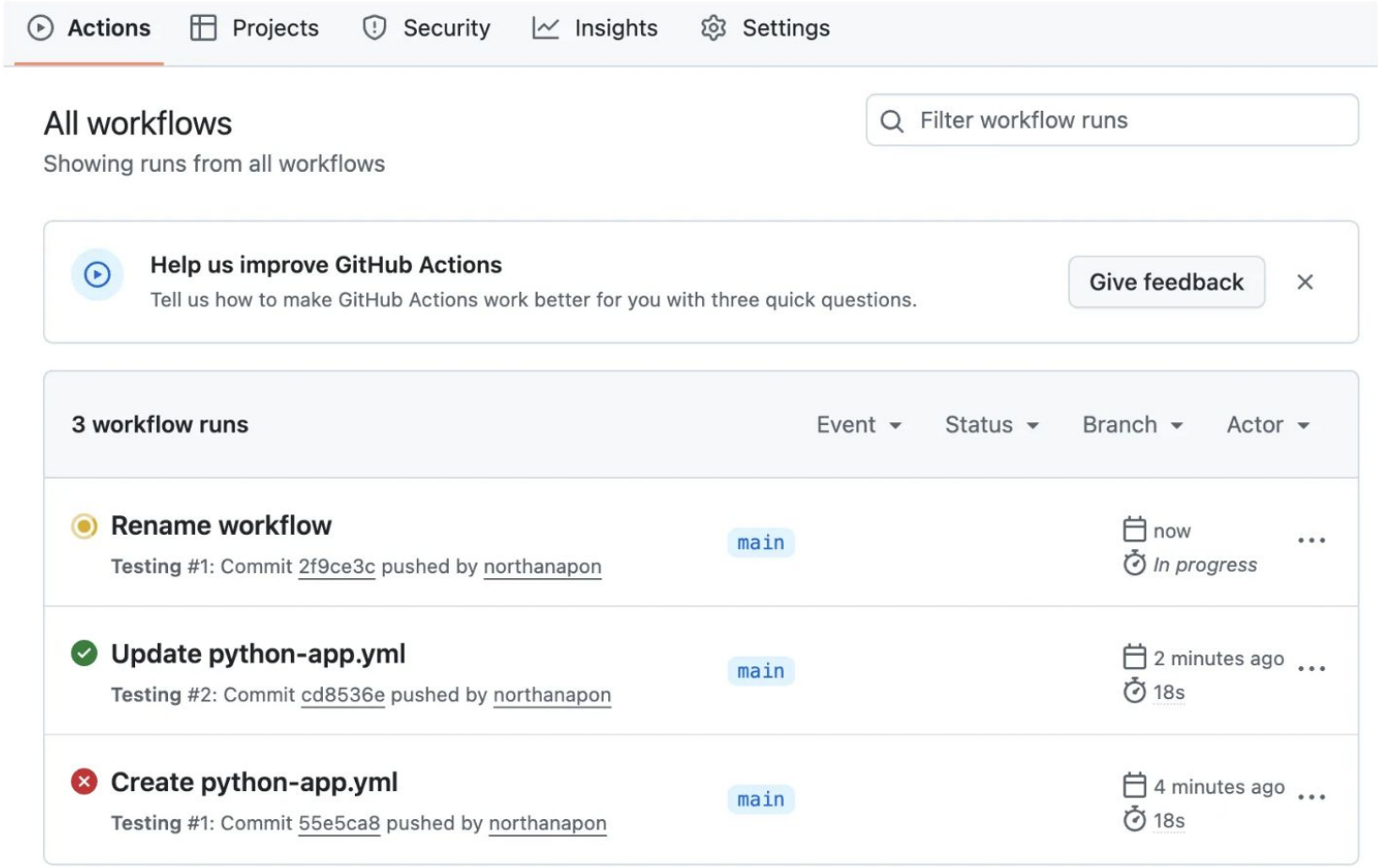
```yaml
key1: value1
key2: value2
key3:
    - item1
    - item2
    - item3
```

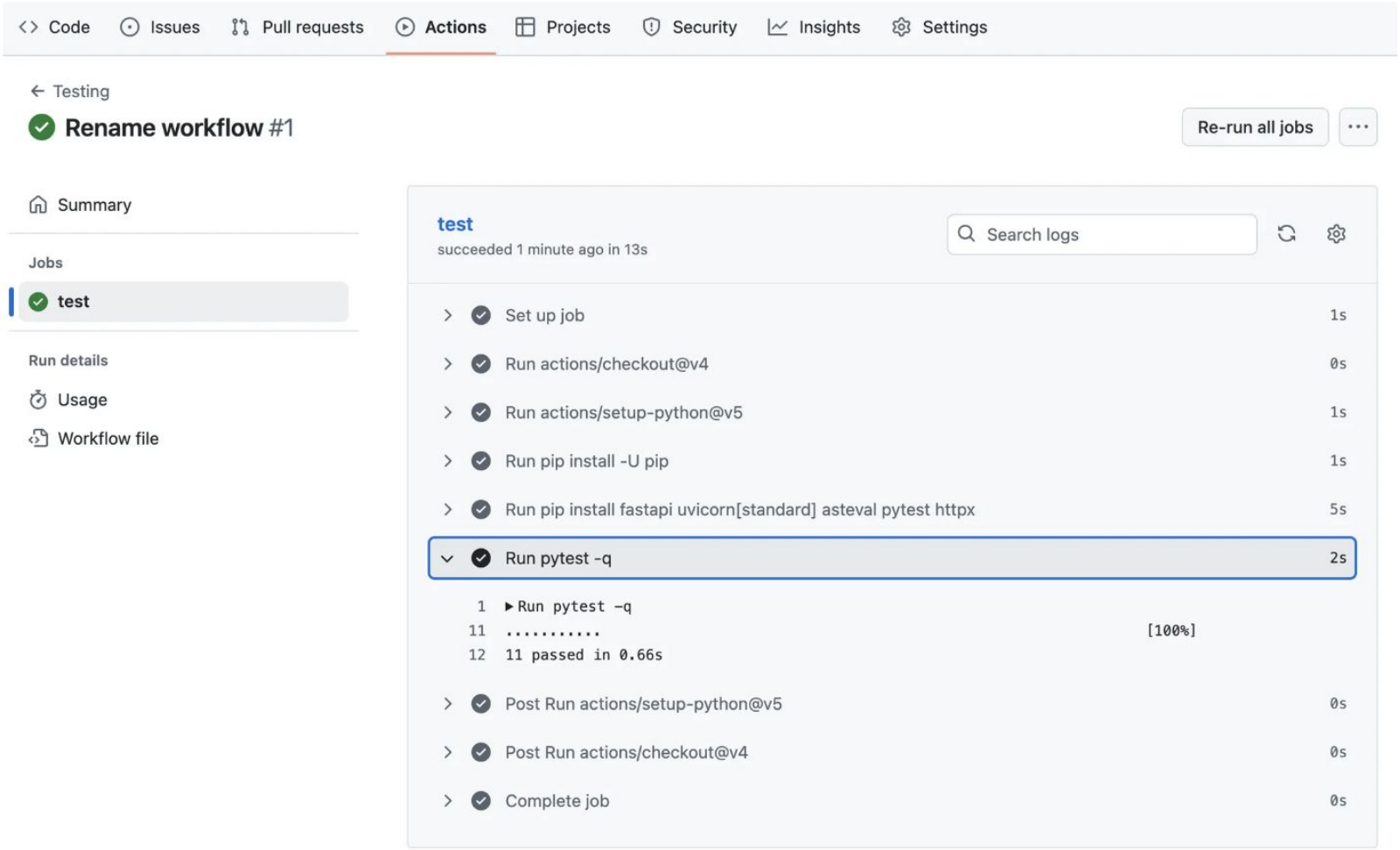
Simple `pytest` Workflow

```yaml
name: Testing
on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -U pip
      - run: pip install fastapi uvicorn[standard] asteval pytest httpx
      - run: pytest -q
```

Result:


Actions
Projects
Security
Insights
Settings
All workflows
Showing runs from all workflows
Filter workflow runs
Help us improve GitHub Actions
Tell us how to make GitHub Actions work better for you with three quick questions.
Give feedback
3 workflow runs
Event
Status
Branch
Actor
Rename workflow
Testing #1: Commit 2f9ce3c pushed by northanapon
main
now
In progress
Update python-app.yml
Testing #2: Commit cd8536e pushed by northanapon
main
2 minutes ago
18s
Create python-app.yml
Testing #1: Commit 55e5ca8 pushed by northanapon
main
4 minutes ago
18s


Code
Issues
Pull requests
Actions
Projects
Security
Insights
Settings
Testing
Rename workflow #1
Re-run all jobs
Summary
Jobs
test
Run details
Usage
Workflow file
test
succeeded 1 minute ago in 13s
Search logs
Set up job 1s
Run actions/checkout@v4 0s
Run actions/setup-python@v5 1s
Run pip install -U pip 1s
Run pip install fastapi uvicorn[standard] asteval pytest httpx 5s
Run pytest -q 2s
1 ▶Run pytest -q
11 ..... [100%]
12 11 passed in 0.66s
Post Run actions/setup-python@v5 0s
Post Run actions/checkout@v4 0s
Complete job 0s

Explanation

Since this is our first time using GitHub Actions, let's go through what each part of this file is doing:

- First, we give the workflow a `name`, which in our case is Testing.
- Next, with the `on` key, we specify when the workflow should run. In our case, we would perform the tests every time someone pushes to the repository.
- The rest of the file is contained within a `jobs` key, which indicates which jobs should be run at every push.
    - In our case, the only job is `test_project`. Every job must define two components
        - The `runs-on` The key specifies which of GitHub's virtual machines we would like our code to be run on.
        - The `steps` key provides the actions that should occur when this job is run
            - In the `uses` key we specify which GitHub action we wish to use. `actions/checkout@v4` is an action written by GitHub that we can use.
            - The `name` key here allows us to provide a description of the action we're taking
            - After the `run` key, we type the commands we wish to run on GitHub's server. In our case, we want to install and run the testing file.